

ECE 20875 Mini Project – Path 2

Team Members: Mary Francis, Prisha Boreddy

Purdue Usernames: franci92, pboredd

Repository Link: <https://github.com/ECEDataScience/project-s25-PrishaB.git>

Dataset

We are given a dataset that measures bike traffic across four bridges in New York City. The data comes from the following bridges: Brooklyn Bridge, Manhattan Bridge, Queensboro Bridge, Williamsburg Bridge. The dataset contains information over the months April through October. This data includes the date, the day of the week, the high/low temperature, the precipitation, the number of bicyclists at each bridge, and the total number of bicyclists for each day.

```
1 Date,Day,High Temp,Low Temp,Precipitation,Brooklyn Bridge,Manhattan Bridge,Williamsburg Bridge,Queensboro Bridge,Total
2 1-Apr,Friday,78.1,66.0,0.01,"1,704","3,126","4,115","2,552","11,497"
3 2-Apr,Saturday,55.48,9.0,0.15,827,"1,646","2,565","1,884","6,922"
4 3-Apr,Sunday,39.9,34.0,0.09,526,"1,232","1,695","1,306","4,759"
5 4-Apr,Monday,44.1,33.1,0.47,521,"1,067","1,440","1,307","4,335"
6 5-Apr,Tuesday,42.1,26.1,0,"1,416","2,617","3,081","2,357","9,471"
7 6-Apr,Wednesday,45.30,0,"1,885","3,329","3,856","2,849","11,919"
8 7-Apr,Thursday,57.53,1.0,0.09,"1,276","2,581","3,282","2,457","9,596"
9 8-Apr,Friday,46.9,44.1,0.01,"1,982","3,455","4,113","3,194","12,744"
10 9-Apr,Saturday,43.37,9.0,0.09,504,997,"1,507","1,502","4,510"
11 10-Apr,Sunday,48.9,30.9,0,"1,447","2,387","3,132","2,160","9,126"
12 11-Apr,Monday,62.1,46.0,0.01,"2,005","3,791","4,334","3,182","13,312"
13 12-Apr,Tuesday,57.45,0.2,"1,045","2,178","2,762","2,032","8,067"
14 13-Apr,Wednesday,57.39,9.0,"2,840","5,395","5,995","4,192","18,422"
15 14-Apr,Thursday,62.1,44.6,0,"2,861","5,389","6,030","4,115","18,315"
16 15-Apr,Friday,64.44,1.0,"2,770","5,072","5,816","3,912","17,570"
17 16-Apr,Saturday,66.45,0,"2,384","4,316","5,624","4,051","16,375"
18 17-Apr,Sunday,73.9,46.0,"3,147","4,969","5,867","4,197","18,180"
19 18-Apr,Monday,81.52,0,"3,871","6,823","7,432","4,964","23,090"
20 19-Apr,Tuesday,71.1,63.0,"3,501","6,951","7,834","5,832","23,318"
21 20-Apr,Wednesday,68.50,0,"3,450","6,574","7,639","4,928","22,591"
22 21-Apr,Thursday,71.1,50.0,"3,436","6,452","7,426","4,813","22,127"
23 22-Apr,Friday,78.1,63.0,"2,975","4,907","6,093","3,862","17,837"
24 23-Apr,Saturday,70.61,0.16,"2,055","3,276","4,856","3,239","13,426"
25 24-Apr,Sunday,68.48,0,"2,798","4,650","5,335","3,957","16,740"
26 25-Apr,Monday,66.9,54.0,"3,463","5,978","6,845","4,564","20,850"
27 26-Apr,Tuesday,60.1,46.9,0.24,"1,997","3,520","4,559","2,929","13,005"
28 27-Apr,Wednesday,62.1,46.9,0,"3,343","5,606","6,577","4,388","19,914"
29 28-Apr,Thursday,57.9,48.0,"2,486","4,152","5,336","3,657","15,631"
30 29-Apr,Friday,57.46,9.0,0.05,"2,375","4,178","5,053","3,348","14,954"
31 30-Apr,Saturday,64.48,0,"3,199","4,952","5,675","3,606","17,432"
32 1-May,Sunday,50.45,0.16,"2,634","1,525","2,062","1,408","7,629"
33 2-May,Monday,59.46,0.04,"2,505","4,149","4,864","3,575","15,093"
34 3-May,Tuesday,54.51,1.0,0.61,"1,059","1,986","2,614","1,741","7,400"
35 4-May,Wednesday,51.1,48.9,0.01,"1,710","2,905","3,797","2,693","11,105"
36 5-May,Thursday,57.46,9.0,"2,582","4,196","4,971","3,619","15,368"
37 6-May,Friday,53.1,48.0,0.54,652,"1,453","2,122","1,612","5,839"
38 7-May,Saturday,57.9,48.9,0,"1,867","3,157","3,789","2,749","11,562"
39 8-May,Sunday,64.9,30.0,0.16,"1,916","2,927","3,940","2,463","11,254"
40 9-May,Monday,72.55,0,"3,650","6,591","7,386","4,951","22,578"
41 10-May,Tuesday,62.1,41.1,0,"3,384","6,283","7,335","4,796","21,798"
42 11-May,Wednesday,72.9,33.1,0,"3,883","7,216","8,231","5,327","24,657"
43 12-May,Thursday,79.57,9.0,"3,960","7,183","8,333","5,735","25,131"
44 13-May,Friday,64.9,60.1,0.25,"1,780","3,072","4,229","3,121","12,202"
45 14-May,Saturday,71.6,60.8,0,"2,556","4,042","5,479","3,951","16,028"
```

With this data, we are given three tasks to complete:

- 1) Given that only three of the four bridges can have a sensor installed on them, determine which three should have sensors monitoring traffic flow in order to best determine the overall traffic.
- 2) Use the next day's weather forecast (low/high temperature and precipitation) to predict the total number of bicyclists for that day.
- 3) Analyze and visualize data to identify any patterns or trends associated with specific days of the week so that you can predict what day it is based on the number of bicyclists on the bridges.

Through this data, we were able to come up with a solution to each of these three tasks.

Data Analysis + Results

Task 1 – Which Three Bridges Should be Used?

The first task we need to tackle is determining which three bridges should be given a sensor. Since we are trying to determine the reliability of each bridge combination in predicting overall traffic, we can find every unique combination of three bridges and use a regression model to predict what the sensors will measure.

```
#making combo options for the bridges
bridges = ['Brooklyn Bridge', 'Manhattan Bridge', 'Queensboro Bridge', 'Williamsburg Bridge']
combos = [
    ['Brooklyn Bridge', 'Manhattan Bridge', 'Williamsburg Bridge'],
    ['Brooklyn Bridge', 'Manhattan Bridge', 'Queensboro Bridge'],
    ['Brooklyn Bridge', 'Williamsburg Bridge', 'Queensboro Bridge'],
    ['Manhattan Bridge', 'Williamsburg Bridge', 'Queensboro Bridge']
]
combo_names = [
    'Brooklyn + Manhattan + Williamsburg',
    'Brooklyn + Manhattan + Queensboro',
    'Brooklyn + Williamsburg + Queensboro',
    'Manhattan + Williamsburg + Queensboro'
]
```

We decided to use a regression model since the data is numeric and we are trying to understand the relationship between total bikers and which combination of three bridges have a similar total biker average to the total averages of the four bridges. We decided to use a K-Nearest Neighbors (KNN) model due to the simplicity of its algorithm and its ability to handle non-linear relationships, which is good for this data set since total bicyclists vs factors like day and weather isn't necessarily always a linear relationship.

```
def get_model(name, params):
    if name == "KNN":
        k = params
        model = KNeighborsRegressor(n_neighbors=k) #regressor in place of classifier from hw10?
    else:
        print("ERROR: Model name not recognized/supported. Returned None")
        model = None
    return model

def get_model_results(model_name, params, train_data, train_labels, test_data, test_labels, num_class):
    model = get_model(model_name, params)

    model.fit(train_data, train_labels)
    test_data_new = model.predict(test_data)
    mse = metrics.mean_squared_error(test_labels, test_data_new)
    # acc = metrics.accuracy_score(test_labels, test_data_new)
    # conf_mat = conf_matrix(test_labels, test_data_new, num_class)
    return mse, model # Return both MSE and model
```

After using the model to calculate predicted values, we can compare those values with the actual data by calculating the RMSE values for each unique combination of three bridges and using the combination with the lowest RMSE value. We can then take the data from the three bridges with the lowest RMSE value and calculate the data's R^2 value to determine just how much of the variation in traffic is actually captured by the model.

```

best_rmse = float('inf')
best_combo = None

for i in range(4): # Iterating through different combos
    X = dataset_2[combos[i]]
    y = dataset_2['total']

    # Split into train/test //using sklearn package for this here?
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model_name = "KNN"
    params = 5
    mse, model = get_model_results(model_name, params, X_train, y_train, X_test, y_test, len(np.unique(y)))

    rmse = np.sqrt(mse)
    print(f"RMSE for {combo_names[i]}: {rmse:.2f}")

    if rmse < best_rmse:
        best_rmse = rmse
        best_combo = combo_names[i]

print(f"\nBest bridge combination to install sensors: {best_combo}")
print(f"Lowest RMSE: {best_rmse:.2f}")
r2 = metrics.r2_score(y_test, model.predict(X_test))
print(f"R²: {r2:.2f}")

```

The RMSE values of each combination is a great way to determine the best three bridges to place the sensors at because it quantifies the average prediction error amongst each subset of bridge sensors. This makes it easier for us to interpret the error measure. Additionally, RMSE is extremely accurate because it emphasizes large errors and allows for fair comparison of the different sensor setups. By looking at the combination with the lowest RMSE yields, we can find the best combination of bridges that allow for reliable traffic estimation. Once we do that, we can look at the R^2 value to determine how accurately the model captures the variation in traffic. A high R^2 value allows us to justify using the bridge combination with the lowest RMSE value since it would stress the accuracy of the KNN model's prediction.

```

RMSE for Brooklyn + Manhattan + Williamsburg: 550.97
RMSE for Brooklyn + Manhattan + Queensboro: 598.35
RMSE for Brooklyn + Williamsburg + Queensboro: 897.98
RMSE for Manhattan + Williamsburg + Queensboro: 925.76

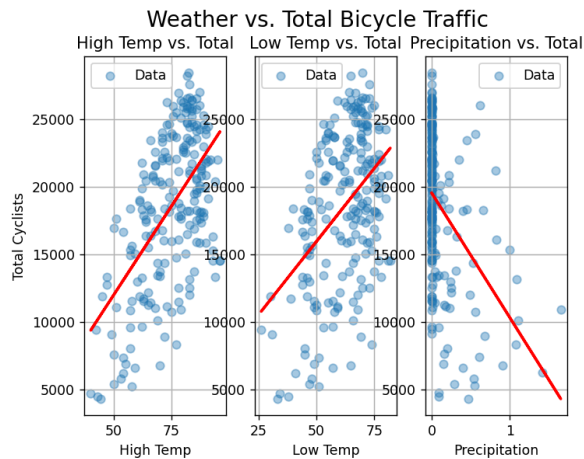
Best bridge combination to install sensors: Brooklyn + Manhattan + Williamsburg
Lowest RMSE: 550.97
R²: 0.98

```

Based on the calculated data, the best bridge combination to install sensors is at the Brooklyn, Manhattan, and Williamsburg bridges, since they have the lowest RMSE value of 550.97. Additionally, the R^2 value of 0.98 is extremely high and suggests that 98% of the traffic variation is justified by the KNN model. This suggests that those three bridges will have the most reliable traffic estimation and we should place the sensors there.

Task 2 – Can we Predict the Total Number of Cyclists Using the Next Day's Weather Forecast?

The next task requires us to use the next day's weather forecast (low/high temperature and precipitation) to predict the total number of bicyclists for that day. Before even calculating anything, we can create a graph of the data to see if there is a correlation between weather and traffic.



As seen in the graphs, there is a positive correlation between temperature and cyclist traffic but a negative correlation between precipitation and cyclist traffic. Additionally, there is a relatively strong correlation for all three of the graphs. However, there are also a good amount of data points that stray away from the line so it would be beneficial to perform a more thorough analysis.

Just like in the previous task, we decided to use a regression model, specifically a KNN regression model, to predict the data. Again, the reasoning behind using this model was identical. This approach also made the most sense since task two is asking us to make predictions using the data anyway. With this approach, we can make predictions and compare with the data to determine the accuracy and if the weather forecast really can predict the total number of cyclists on a given day.

```
# Part 2

#cleaning up the csv
dataset_2['High Temp'] = pd.to_numeric(dataset_2['High Temp'])
dataset_2['Low Temp'] = pd.to_numeric(dataset_2['Low Temp'])
dataset_2['Precipitation'] = pd.to_numeric(dataset_2['Precipitation'])
# weather features to predict total traffic
X_forecast = dataset_2[['High Temp', 'Low Temp', 'Precipitation']]
y_forecast = dataset_2['Total']

# using train/test split
X_train, X_test, y_train, y_test = train_test_split(X_forecast, y_forecast, test_size=0.2, random_state=42)

# knn regression
model_name = "KNN"
params = 5
mse, model = get_model_results(model_name, params, X_train, y_train, X_test, y_test, len(np.unique(y)))
rmse = np.sqrt(mse)

#try computing r^2 metric instead we want ~0.5 (according to piazza)

print("\nWeather Forecast Prediction ")

r2 = metrics.r2_score(y_test, model.predict(X_test))
print(f"R2: {r2:.2f}")
print(f"RMSE: {rmse:.2f}")
```

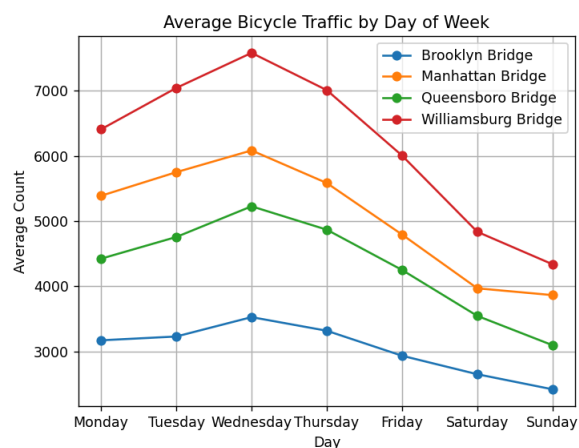
After predicting values with the KNN model, we calculated the RMSE value of the weather predictions compared to the given data as well as the R^2 value of model data. The output we got is given below.

```
Weather Forecast Prediction
R2: 0.46
RMSE: 4716.91
```

Given that the model determined an R^2 score of 0.46, which is extremely close to the recommended 0.5, the model is fairly good at predicting the next day cyclist total based on weather. However, it is only average at doing so since the value means that the model only explains about 46% of the variation in total bike traffic based on weather features. Additionally, the RMSE value of 4716.91 is pretty high and suggests that the prediction is not the most accurate and can be a little off from the actual total cyclist averages. This suggests that while it is possible to use the weather to determine the next day's total number of cyclists, this prediction will not always be the most reliable and could be off due to other factors (such as events, holidays, etc).

Task 3 – What Patterns or Trends Associated with Specific Days of the Week Can be Used to Predict What Day it is Based on the Number of Bicyclists on the Bridges

The final task required us to analyze and visualize data to identify any patterns or trends associated with specific days of the week so that you can predict what day it is based on the number of bicyclists on the bridges. To start, we can first generate graphs of the data to get a visible visualization of the data. Doing this allows us to notice any noticeable trends before using algorithms to get a more accurate read.



As seen above in the graph which displays the average traffic by day of week at each of the four bridges, there is a general trend where the weekend has the least amount of bicycle traffic. Additionally, we can see that there is a spike during the middle of the week with Wednesday

having the most traffic out of all the days of the week. Before doing any other calculations, we can see that it is possible to determine the day of the week based on the total traffic of the day but it might be difficult for days like Tuesday and Thursday, since their days have very similar averages across all the bridges.

Below is the numerical representation of what is presented in the graph:

Average bridge traffic per day:				
	Brooklyn Bridge	Manhattan Bridge	Queensboro Bridge	Williamsburg Bridge
Day				
Friday	2933.838710	4793.935484	4250.354839	6006.451613
Monday	3170.741935	5389.419355	4423.709677	6409.838710
Saturday	2650.000000	3968.903226	3545.548387	4836.193548
Sunday	2418.580645	3864.580645	3095.451613	4337.774194
Thursday	3317.466667	5584.866667	4868.566667	7010.400000
Tuesday	3229.400000	5751.533333	4756.700000	7044.633333
Wednesday	3526.533333	6085.466667	5227.666667	7582.600000

In order to further determine the accuracy of the calculations, we decided to use a KNN classifier to learn the patterns in bridge traffic and associate them with the correct day of the week. We used a classifier model this time instead of a regression model since the data we are predicting is categorical and not numerical (days of the week vs total number of bicycle traffic). Classifier models are used when the predicted value is categorical so that's why it was the best choice for this task.

By using the KNN classifier, we can train it to notice the trends in average bicyclist traffic vs the day of the week. This model can then be used to make predictions based on the data it received. Once that is complete, the model's predictions can be compared with the actual data and be used to determine the accuracy of using traffic to determine the day of the week.

```
# Part 3
#cleaning up the csv
# from datetime import datetime

dataset_2['Day of Week'] = dataset_2['Day']

#grouping by day
#and averaging for each bridge
avg_by_day = dataset_2.groupby('Day')[['Brooklyn Bridge', 'Manhattan Bridge', 'Queensboro Bridge', 'Williamsburg Bridge']].mean()

print("Average bridge traffic per day:")
print(avg_by_day)

# Prepare features and target
X_day = dataset_2[['Brooklyn Bridge', 'Manhattan Bridge', 'Queensboro Bridge', 'Williamsburg Bridge']]
y_day = dataset_2['Day of Week']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_day, y_day, test_size=0.2, random_state=42)

# KNN classifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred_day = model.predict(X_test)
accuracy = metrics.accuracy_score(y_test, y_pred_day)
print(f"Accuracy in predicting the day of the week: {accuracy * 100:.2f}%")

conf_matrix = conf_matrix(y_test, y_pred_day, np.unique(y_day))
```

The following is the accuracy of the model in predicting the day of the week based on the traffic:

Accuracy in predicting the day of the week: 20.93%

Given that the value is 20.93% and that the likelihood of guessing something right by chance is roughly 17%, the model is incredibly inaccurate. This suggests the model predicted 79.07% of the days incorrectly and supports the deduction presented by the graph and table above. Using traffic to predict the day of the week is not a good idea and this makes sense since other factors can also influence traffic (such as weather conditions and if any events are happening).